Universidad Nacional de Río Cuarto

Sistemas Operativos

(Código 1965)

Taller 1: Arquitectura RISC-V

Kernel mínimo en M-mode y S-mode con manejo de traps

Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Alumno
Tomás Rodeghiero

Fecha de entrega: 19 de abril de 2026

Índice

1. Objetivo del trabajo **2**

2. Entorno de desarrollo **2**

2.1. Herramientas utilizadas 2

2.2. Estructura de la entrega 2

3. Parte 1 - Hello World en M-mode **3**

3.1. Descripción 3

3.2. Compilación y ejecución 3

3.3. Salida obtenida 3

3.4. Captura de pantalla mostrando el kernel corriendo y el código fuente del archivo kernel.c 3

3.5. Código fuente 4

3.6. Resolución de los ejercicios del README de `00-hello-world` 5

3.6.1. Ejercicio 1: listado de símbolos con `nm` 5

3.6.2. Ejercicio 2: análisis del archivo `kernel.asm` 5

3.6.3. Ejercicio 3: inspección ELF con `readelf` 6

3.6.4. Ejercicio 4: contenido hexadecimal de `.text` 9

3.6.5. Ejercicio 5: contenido hexadecimal de `.rodata` 9

3.6.6. Ejercicio 6: verificación de ejecución en M-mode mediante `mhartid` . 10

4. Parte 2 - S-mode y manejo de traps **11**

4.1. Contexto 11

4.2. Cómo se invoca `trap(cause)` desde `start.s` 11

4.3. Ejecución base 11

4.4. Prueba comentando la primera línea de `kernel_main()` 12

4.5. Respuesta conceptual (paso 6) 12

4.6. Implementación del TO DO en `trap()` 12

4.7. Código fuente final del archivo kernel.c 13

5. Conclusiones **15**

Objetivo del trabajo

Este taller tuvo como objetivo entender, con una práctica concreta, cómo arranca y se ejecuta un kernel mínimo en RISC-V bajo distintos niveles de privilegio. La idea no fue solo compilar y correr ejemplos, sino mirar qué pasa en cada etapa del boot y por qué.

El trabajo se organizó en dos partes complementarias:

- **Parte 1:** validación del arranque básico en *Machine Mode* (M-mode), compilación del proyecto y salida por UART.
- **Parte 2:** transición a *Supervisor Mode* (S-mode), análisis del mecanismo de traps y resolución del `TO DO` dentro de `trap()`.

En términos de aprendizaje, el foco estuvo en relacionar el código con el comportamiento real del procesador: qué instrucciones son legales en cada modo, cómo se reporta una excepción y de qué manera se puede diagnosticar desde el kernel.

Entorno de desarrollo

2.1 Herramientas utilizadas

- Emulador: `qemu-system-riscv32` (QEMU 10.2.2).
- Toolchain: `riscv64-unknown-elf-gcc` (GCC 15.1.0).
- Sistema de compilación: `make` con `PREFIX=riscv64-unknown-elf-`.

2.2 Estructura de la entrega

- `00-hello-world/`: código y pruebas de la Parte 1.
- `01-hello-world-S-mode/`: código y pruebas de la Parte 2.
- `README.md`: guía general del taller.
- `entrega/`: carpeta con los archivos finales de la entrega.
- `informe-taller1-riscv.pdf`: informe final en PDF.

Parte 1 - Hello World en M-mode

3.1 Descripción

En este primer paso se trabaja con un kernel muy simple que inicia en M-mode y escribe por consola serie usando UART mapeada en memoria. Es el punto de partida para comprobar que la herramienta de compilación, el linker script y la ejecución sobre QEMU están funcionando como se espera.

3.2 Compilación y ejecución

Listing 3.1: Comandos utilizados en la Parte 1

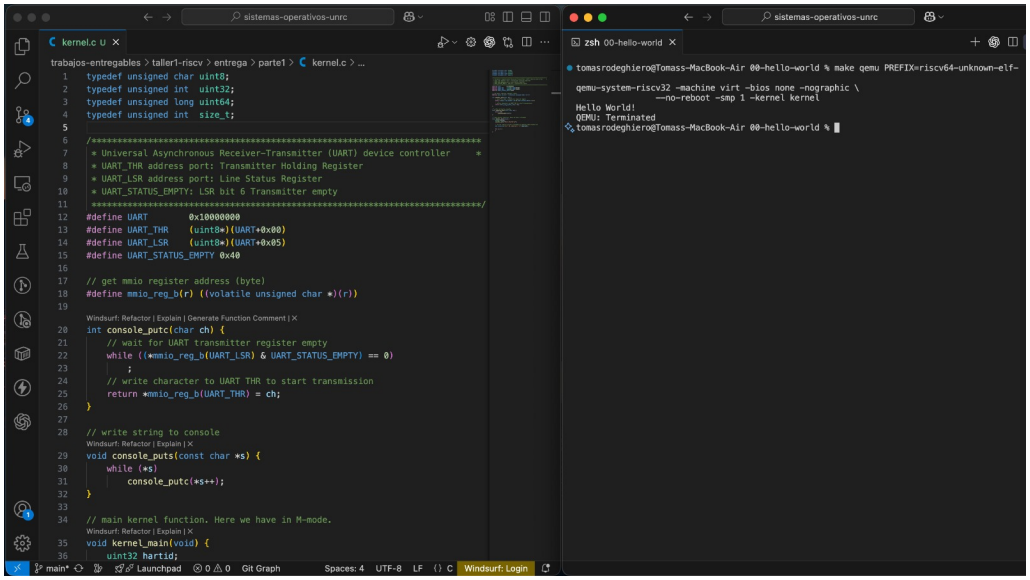
```
cd 00-hello-world
make PREFIX=riscv64-unknown-elf-
make qemu PREFIX=riscv64-unknown-elf-
```

3.3 Salida obtenida

```
Hello World!
```

La salida observada fue la esperada: `Hello World!`.

3.4 Captura de pantalla mostrando el kernel corriendo y el código fuente del archivo kernel.c



3.5 Código fuente

kernel.c - Parte 1

```c
typedef unsigned char uint8;
typedef unsigned int  uint32;
typedef unsigned long uint64;
typedef unsigned int  size_t;

/*************************************************************************
 * Universal Asynchronous Receiver-Transmitter (UART) device controller     *
 * UART_THR address port: Transmitter Holding Register
 * UART_LSR address port: Line Status Register
 * UART_STATUS_EMPTY: LSR bit 6 Transmitter empty
 *************************************************************************/
#define UART        0x10000000
#define UART_THR    (uint8*)(UART+0x00)
#define UART_LSR    (uint8*)(UART+0x05)
#define UART_STATUS_EMPTY 0x40

// get mmio register address (byte)
#define mmio_reg_b(r) ((volatile unsigned char *)(r))

int console_putc(char ch) {
    // wait for UART transmitter register empty
    while ((*mmio_reg_b(UART_LSR) & UART_STATUS_EMPTY) == 0)
        ;
    // write character to UART THR to start transmission
    return *mmio_reg_b(UART_THR) = ch;
}

// write string to console
void console_puts(const char *s) {
    while (*s)
        console_putc(*s++);
}

// main kernel function. Here we have in M-mode.
void kernel_main(void) {
    uint32 hartid;
    console_puts("Hello World!\n");

    // to do: verify we are in M-mode by reading some m-mode CSR
    asm volatile("csrr %0, mhartid": "=r"(hartid));

    for (;;) {
    }
}
```

3.6 Resolución de los ejercicios del README de `00-hello-world`

Como complemento de la ejecución base del kernel, se resolvieron los ejercicios técnicos propuestos en el README de la carpeta `00-hello-world`. Esta sección documenta el procedimiento, la evidencia y la interpretación de cada punto, manteniendo el enfoque en la relación entre código fuente, binario enlazado y comportamiento de ejecución.

3.6.1 Ejercicio 1: listado de símbolos con `nm`

Para inspeccionar los símbolos del binario y su orden en memoria, se ejecutó:

Listado de símbolos ordenados por dirección

```
riscv64-unknown-elf-nm -n kernel
```

Salida completa:

```
80000000 T boot
8000000c T console_putc
80000048 T console_puts
8000008c T kernel_main
800000c0 R __bss
800000c0 R __bss_end
80001000 R __stack0
80002000 R __kernel_end
```

Resultados observados:

- `boot` en `0x80000000`.
- `kernel_main` en `0x8000008c`.
- `__stack0` en `0x80001000`.
- `__kernel_end` en `0x80002000`.

La diferencia entre `__stack0` y `__kernel_end` es `0x1000` (4 KB), coherente con la reserva de stack definida por el linker script. Además, los tipos de símbolo (`T`, `R`, etc.) permiten distinguir código, datos y símbolos de enlace.

3.6.2 Ejercicio 2: análisis del archivo `kernel.asm`

Se analizó el desensamblado generado por `objdump` en `kernel.asm`. A partir de ese análisis se verificó:

- La rutina de arranque (`boot`) inicializa el stack y deriva el control a `kernel_main`.
- `kernel_main` imprime el mensaje por UART y permanece en un ciclo infinito.

- `console_puts` recorre la cadena y delega el envío carácter por carácter en `console_putc`.
- `console_putc` espera el bit de disponibilidad del transmisor en `UART_LSR` y luego escribe en `UART_THR`.

Este punto permitió conectar el código C con su traducción efectiva a instrucciones RISC-V.

3.6.3 Ejercicio 3: inspección ELF con `readelf`

Se ejecutó:

Inspección de metadatos ELF

```
riscv64-unknown-elf-readelf -at kernel
```

Salida completa:

```
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           RISC-V
  Version:                           0x1
  Entry point address:               0x80000000
  Start of program headers:          52 (bytes into file)
  Start of section headers:          6688 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         3
  Size of section headers:           40 (bytes)
  Number of section headers:         16
  Section header string table index: 15

Section Headers:
  [Nr] Name
       Type            Addr     Off    Size   ES   Lk Inf Al
       Flags
  [ 0]
       NULL            00000000 000000 000000 00   0   0  0
       [00000000]:
  [ 1] .text
       PROGBITS        80000000 001000 0000b0 00   0   0  4
       [00000006]: ALLOC, EXEC
  [ 2] .rodata
       PROGBITS        800000b0 0010b0 00000e 01   0   0  4
       [00000032]: ALLOC, MERGE, STRINGS
  [ 3] .riscv.attributes
```

```
       RISCV_ATTRIBUTE 00000000 0010be 00004f 00   0   0  1
       [00000000]:
  [ 4] .comment
       PROGBITS        00000000 00110d 000012 01   0   0  1
       [00000030]: MERGE, STRINGS
  [ 5] .debug_line
       PROGBITS        00000000 00111f 000152 00   0   0  1
       [00000000]:
  [ 6] .debug_line_str
       PROGBITS        00000000 001271 000061 01   0   0  1
       [00000030]: MERGE, STRINGS
  [ 7] .debug_info
       PROGBITS        00000000 0012d2 00011e 00   0   0  1
       [00000000]:
  [ 8] .debug_abbrev
       PROGBITS        00000000 0013f0 0000ee 00   0   0  1
       [00000000]:
  [ 9] .debug_aranges
       PROGBITS        00000000 0014e0 000040 00   0   0  8
       [00000000]:
  [10] .debug_str
       PROGBITS        00000000 001520 00017e 01   0   0  1
       [00000030]: MERGE, STRINGS
  [11] .debug_loclists
       PROGBITS        00000000 00169e 00001b 00   0   0  1
       [00000000]:
  [12] .debug_frame
       PROGBITS        00000000 0016bc 000080 00   0   0  4
       [00000000]:
  [13] .symtab
       SYMTAB          00000000 00173c 000190 10  14  17  4
       [00000000]:
  [14] .strtab
       STRTAB          00000000 0018cc 0000a0 00   0   0  1
       [00000000]:
  [15] .shstrtab
       STRTAB          00000000 00196c 0000b1 00   0   0  1
       [00000000]:

There are no section groups in this file.

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  RISCV_ATTRIBUT 0x0010be 0x00000000 0x00000000 0x0004f 0x00000 R   0x1
  LOAD           0x001000 0x80000000 0x80000000 0x000be 0x000be R E 0x1000
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RW  0x10

 Section to Segment mapping:
  Segment Sections...
   00     .riscv.attributes
   01     .text .rodata
   02

There is no dynamic section in this file.

There are no relocations in this file.
```

```
The decoding of unwind sections for machine type RISC-V is not currently supported.

Symbol table '.symtab' contains 25 entries:
   Num:    Value  Size Type    Bind   Vis      Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 80000000     0 SECTION LOCAL  DEFAULT    1 .text
     2: 800000b0     0 SECTION LOCAL  DEFAULT    2 .rodata
     3: 00000000     0 SECTION LOCAL  DEFAULT    3 .riscv.attributes
     4: 00000000     0 SECTION LOCAL  DEFAULT    4 .comment
     5: 00000000     0 SECTION LOCAL  DEFAULT    5 .debug_line
     6: 00000000     0 SECTION LOCAL  DEFAULT    6 .debug_line_str
     7: 00000000     0 SECTION LOCAL  DEFAULT    7 .debug_info
     8: 00000000     0 SECTION LOCAL  DEFAULT    8 .debug_abbrev
     9: 00000000     0 SECTION LOCAL  DEFAULT    9 .debug_aranges
    10: 00000000     0 SECTION LOCAL  DEFAULT   10 .debug_str
    11: 00000000     0 SECTION LOCAL  DEFAULT   11 .debug_loclists
    12: 00000000     0 SECTION LOCAL  DEFAULT   12 .debug_frame
    13: 00000000     0 FILE    LOCAL  DEFAULT  ABS ccgiG2fq.o
    14: 80000000     0 NOTYPE  LOCAL  DEFAULT    1 $xrv32i2p1_m2p0_[...]
    15: 00000000     0 FILE    LOCAL  DEFAULT  ABS kernel.c
    16: 8000000c     0 NOTYPE  LOCAL  DEFAULT    1 $xrv32i2p1_m2p0_[...]
    17: 80000048    68 FUNC    GLOBAL DEFAULT    1 console_puts
    18: 80002000     0 NOTYPE  GLOBAL DEFAULT    2 __kernel_end
    19: 80000000     0 NOTYPE  GLOBAL DEFAULT    1 boot
    20: 800000c0     0 NOTYPE  GLOBAL DEFAULT    2 __bss_end
    21: 8000000c    60 FUNC    GLOBAL DEFAULT    1 console_putc
    22: 8000008c    36 FUNC    GLOBAL DEFAULT    1 kernel_main
    23: 800000c0     0 NOTYPE  GLOBAL DEFAULT    2 __bss
    24: 80001000     0 NOTYPE  GLOBAL DEFAULT    2 __stack0

No version information found in this file.
Attribute Section: riscv
File Attributes
  Tag_RISCV_stack_align: 16-bytes
  Tag_RISCV_arch: "rv32i2p1_m2p0_a2p1_zicsr2p0_zmmul1p0_zaamo1p0_zalrsc1p0"
  Tag_RISCV_priv_spec: 1
  Tag_RISCV_priv_spec_minor: 11
```

Verificaciones principales:

- Tipo de archivo: `EXEC (Executable file)`.
- Arquitectura objetivo: `RISC-V`.
- Punto de entrada: `0x80000000`.
- Secciones relevantes presentes: `.text` y `.rodata`.

Los datos coinciden con lo esperado para un kernel mínimo enlazado en la dirección de arranque definida por `kernel.ld`.

3.6.4 Ejercicio 4: contenido hexadecimal de `.text`

Para visualizar el contenido de la sección de código ejecutable, se ejecutó:

Hexdump de la sección .text

```
riscv64-unknown-elf-objdump -s -j .text kernel
```

Salida completa:

```
kernel:     file format elf32-littleriscv

Contents of section .text:
 80000000 17210000 13010100 ef004008 130101ff  .!........@.....
 80000010 23261100 23248100 13040101 37070010  #&..#$......7...
 80000020 13075700 83470700 93f70704 e38c07fe  ..W..G..........
 80000030 b7070010 2380a700 8320c100 03248100  ....#.... ...$..
 80000040 13010101 67800000 130101ff 23261100  ....g.......#&..
 80000050 23248100 23229100 13040101 93040500  #$..#".........
 80000060 03450500 630a0500 93841400 eff01ffa  .E..c...........
 80000070 03c50400 e31a05fe 8320c100 03248100  ......... ...$..
 80000080 83244100 13010101 67800000 130101ff  .$A.....g.......
 80000090 23261100 23248100 13040101 17050000  #&..#$..........
 800000a0 13054501 eff05ffa f32740f1 6f000000  ..E..._..'@.o...
```

La salida muestra la representación hexadecimal de las instrucciones máquina que implementan el arranque y las funciones del kernel.

3.6.5 Ejercicio 5: contenido hexadecimal de `.rodata`

Para inspeccionar datos constantes del programa, se ejecutó:

Hexdump de la sección .rodata

```
riscv64-unknown-elf-objdump -s -j .rodata kernel
```

Salida completa:

```
kernel:     file format elf32-littleriscv

Contents of section .rodata:
 800000b0 48656c6c 6f20576f 726c6421 0a00      Hello World!..
```

En esta sección se identifica la cadena `"Hello World!\n"`, lo cual confirma que el literal utilizado por `console_puts` quedó correctamente ubicado en memoria de solo lectura.

3.6.6 Ejercicio 6: verificación de ejecución en M-mode mediante `mhartid`

La verificación se implementó en `kernel.c` con inline assembly:

Lectura del CSR mhartid

```
uint32 hartid;
asm volatile("csrr %0, mhartid": "=r"(hartid));
```

`mhartid` es un CSR (*Control and Status Register*) de machine mode. En esta Parte 1, la lectura se ejecuta sin excepción, lo que es consistente con la ejecución del kernel en M-mode. Esta observación también se alinea con la Parte 2, donde el mismo acceso desde S-mode produce *Illegal Instruction*.

Parte 2 - S-mode y manejo de traps

4.1 Contexto

En esta parte el cambio importante es que el kernel deja de ejecutarse en M-mode y pasa a S-mode durante el boot. Ese detalle modifica qué instrucciones son válidas y, por lo tanto, vuelve relevante el manejo de excepciones.

Además, el proyecto originalmente ya trae armado un esquema de traps en dos niveles:

1. **Una capa en ensamblador (`start.s`)**, donde la rutina `s_trap` captura el evento.
2. **Una capa en C (`kernel.c`)**, donde la función `trap(cause)` interpreta la causa y muestra información útil por UART.

Este diseño separa bien responsabilidades: el ensamblador hace la captura mínima de bajo nivel y C se encarga de la lógica de diagnóstico.

4.2 Cómo se invoca `trap(cause)` desde `start.s`

Invocación desde `s_trap`

```
s_trap:
    csrr a0, scause
    call trap
```

La invocación es directa y sigue la convención de llamada de RISC-V:

- `csrr a0, scause` lee el CSR `scause` y guarda su valor en `a0`.
- En la ABI de RISC-V, `a0` es el registro del primer argumento.
- Luego, `call trap` transfiere el control a `trap(uint cause)` usando ese valor como parámetro `cause`.

En otras palabras, el handler en ensamblador no interpreta la causa: solo la toma del hardware y se la pasa a C para su procesamiento.

4.3 Ejecución base

```
trap!!!
```

4.4 Prueba comentando la primera línea de `kernel_main()`

Se comentó temporalmente:

```
// asm volatile("csrr a0, mhartid");
```

Resultado obtenido:

```
Hello World from supervisor mode!
```

4.5 Respuesta conceptual (paso 6)

La excepción del paso 4 al ejecutar el código aparece porque `kernel_main()` ya está corriendo en S-mode, pero la primera línea intenta leer `mhartid`, que es un CSR privilegiado asociado al modo máquina.

Desde S-mode esa lectura no está permitida. Por eso, cuando la CPU intenta ejecutar esa instrucción, la arquitectura la marca como instrucción ilegal y dispara un trap. El valor `scause = 2` coincide justamente con *Illegal Instruction*, y eso explica por qué al comentar esa línea el problema desaparece.

4.6 Implementación del TO DO en `trap()`

Se implementó:

- Separación entre interrupción y excepción usando el bit más significativo de `scause`.
- Obtención del código de causa en los bits bajos.
- Detección explícita de *Illegal Instruction* (código 2).
- Mensajes de diagnóstico por UART para verificar el comportamiento en ejecución.

Salida final con implementación:

```
trap!!!
trap type: exception
exception code: 2
exception detail: Illegal Instruction
```

4.7 Código fuente final del archivo kernel.c

`kernel.c` final (Parte 2)

```c
typedef unsigned char uint8;
typedef unsigned int  uint32;
typedef unsigned int  uint;
typedef unsigned long uint64;
typedef unsigned int  size_t;

/*****************************************************************************
 * Universal Asynchronous Receiver-Transmitter (UART) device controller     *
 * UART_THR address port: Transmitter Holding Register
 * UART_LSR address port: Line Status Register
 * UART_STATUS_EMPTY: LSR bit 6 Transmitter empty
 *****************************************************************************/
#define UART        0x10000000
#define UART_THR    (uint8*)(UART+0x00)
#define UART_LSR    (uint8*)(UART+0x05)
#define UART_STATUS_EMPTY 0x40
#define SCAUSE_INTERRUPT_MASK (1U << 31)
#define SCAUSE_CODE_MASK      0x7fffffffU
#define SCAUSE_ILLEGAL_INSTRUCTION 2U

// get mmio register address (byte)
#define mmio_reg_b(r) ((volatile unsigned char *)(r))

int console_putc(char ch) {
    // wait for UART transmitter register empty
    while ((*mmio_reg_b(UART_LSR) & UART_STATUS_EMPTY) == 0)
        ;
    // write character to UART THR to start transmission
    return *mmio_reg_b(UART_THR) = ch;
}

// write string to console
void console_puts(const char *s) {
    while (*s)
        console_putc(*s++);
}

void console_putu(uint32 value) {
    char buf[11];
    int i = 0;

    if (value == 0) {
        console_putc('0');
        return;
    }

    while (value > 0) {
        buf[i++] = (char)('0' + (value % 10));
        value /= 10;
    }

    while (i > 0) {
```

```c
        console_putc(buf[--i]);
    }
}

// the trap handler (called from s_trap defined in start.s)
void trap(uint cause)
{
    console_puts("trap!!!\n");

    // Detect trap type and cause from scause:
    // bit 31: 1=interrupt, 0=exception
    // bits 30..0: exception/interrupt code
    uint is_interrupt = (cause & SCAUSE_INTERRUPT_MASK) != 0;
    uint code = cause & SCAUSE_CODE_MASK;

    if (is_interrupt) {
        console_puts("trap type: interrupt\n");
        console_puts("interrupt code: ");
        console_putu(code);
        console_putc('\n');
    } else {
        console_puts("trap type: exception\n");
        console_puts("exception code: ");
        console_putu(code);
        console_putc('\n');

        if (code == SCAUSE_ILLEGAL_INSTRUCTION) {
            console_puts("exception detail: Illegal Instruction\n");
        }
    }

    // In this step we stop execution after reporting the trap.
    for (;;) {
    }

}

const char *hello = "Hello World from supervisor mode!\n";

// main kernel function. Here we have in supervisor (kernel) mode.
void kernel_main(void) {
    asm volatile("csrr a0, mhartid");
    console_puts(hello);
    for (;;) {
    }
}
```

Conclusiones

Este taller me ayudó a entender, en un caso real y acotado, cómo se combinan niveles de privilegio, arranque del kernel y manejo de excepciones en RISC-V.

La Parte 1 me sirvió para validar la base del entorno y la salida por UART. La Parte 2 me agregó lo más interesante desde Sistemas Operativos: ver qué cambia al pasar a S-mode, detectar una excepción genuina por privilegios y resolver el handler para interpretar correctamente `scause`.